Next, we'll discuss about Flask-SQLAlchemy and Flask- MongoEngine and create Flask CRUD applications using them.

Creating a Flask Application with SQLAlchemy

Flask-SQLAlchemy is an extension for flask which adds support for SQLAlchemy to the application. SQLAlchemy is a Python toolkit and Object Relational Mapper that provides access to the SQL database using Python. SQLAlchemy comes with enterprise-level persistence patterns and efficient and high performing database access. Flask-SQLAlchemy provides support for the following SQL-based database engines given the appropriate DBAPI driver is installed:

- PostgreSQL
- MySQL
- Oracle
- SQLite
- Microsoft SQL Server
- Firebird SyBase

We'll be using MySQL as the database engine in our application, so let's get started with installing SQLAlchemy and start setting up our application.

Let's create a new directory called flask-MySQL, create a virtual environment, and then install flask-sqlalchemy.

```
$ mkdir flask-mysql && cd flask-mysql
```

Now, create a virtual environment inside the directory using the following command:

```
$ virtualenv venv
```

As discussed earlier, we can activate the virtual environment using the following command:

```
$ source venv/bin/activate
```

Once the virtual environment is activated, let's install flask-sqlalchemy.

Flask and Flask-SQLAlchemy can be installed using PIP with the following command.

```
(venv)$ pip install flask flask-sqlalchemy
```

Other than SQLite, all other database engines need separate libraries to be installed alongside Flask-SQLAlchemy for it to work. SQLAlchemy uses MySQL-Python as the default DBAPI for connecting with MySQL.

Now, let's install PyMySQL to enable MySQL connection with Flask-SQLAlchemy.

```
(venv) $ pip install pymysql
```

Now, we should have everything we need to create our sample flask-MySQL application with.

Let's start by creating app.py which will contain the code for our application. After creating the file, we'll initiate the Flask application.

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] =
'mysql+pymysql://<mysql_username>:<mysql_password>@<mysql_
host>:<mysql_port>/<mysql_db>'
db = SQLAlchemy(app)

if __name__ == "__main__":
    app.run(debug=True)
```

Here, we import the Flask framework and Flask-SQLAlchemy and then initiate an instance of Flask. After that, we configure the SQLAlchemy database URI to use our MySQL DB URI, and then we create an object of SQLAlchemy named as db, which will handle our ORM-related activities.

Now, if you are using MySQL, make sure you supply connection strings of a running MySQL server and that the database name supplied does exist.

Note Use environment variables for supplying database connection strings in your applications.

Make sure that you have a running MySQL server to follow this application. However, you can also use SQLite in its place by supplying the SQLite config details in the SQLAlchemy database URI which should look like this:

```
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:////tmp/
<db_name>.db'
```

In order to run the application, you need to execute the following code in your terminal:

```
(venv) $ python app.py
```

And if there are no errors, you should see a similar output in your terminal:

```
(venv) $ python app.py
* Serving Flask app "app" (lazy loading)
 * Environment: production
   WARNING: Do not use the development server in a production
   environment.
   Use a production WSGI server instead.
 * Debug mode: on
 * Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

```
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 779-301-240
```

Creating an Author Database

We'll now create an author database application which will provide RESTful CRUD APIs. All the authors will be stored in a table titled "authors".

After the declared db object, add the following lines of code to declare a class as Authors which will hold the schema for the author table:

```
class Author (db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(20))
    specialisation = db.Column(db.String(50))

    def __init__(self, name, specialisation):
        self.name = name
        self.specialisation = specialisation
    def __repr__(self):
      return '<Product %d>' % self.id
db.create_all()
```

With this code, we have created a model titled "Authors" which has three fields—ID, name, and specialisation. Name and specialisation are strings, but ID is a self-generated and auto-incremented integer which will serve as a primary key. Notice the last line "db.create_all()" which instructs the application to create all the tables and database specified in the application.

In order to serve JSON response from our API using the data returned by SQLAlchemy, we need another library called marshmallow which is an add-on to SQLAlchemy to serialize SQLAlchemy-returned data objects to JSON.

```
(venv)$ pip install flask-marshmallow
```

The following command will install the Flask version of marshmallow in our application, and we'll define our output schema from the Authors model using marshmallow.

Add the following lines on the top, below the other imports in your application file to import marshmallow.

```
from marshmallow_sqlalchemy import ModelSchema
from marshmallow import fields
```

After the db.create_all(), define your output schema using the following code:

```
class AuthorSchema(ModelSchema):
    class Meta(ModelSchema.Meta):
        model = Authors
        sqla_session = db.session

    id = fields.Number(dump_only=True)
    name = fields.String(required=True)
    specialisation = fields.String(required=True)
```

The preceding code maps the variable attribute to field objects, and in Meta, we define the model to relate to our schema. So this should help us return JSON from SQLAlchemy.

After setting up our model and return schema, we can jump to creating our endpoints. Let's create our first GET /authors endpoint to return all the registered authors. This endpoint will query for all the objects in the Authors model and return them in JSON to the user. But before we write the endpoint, edit the first import line to the following to import jsonify, make_response, and request from Flask.

```
from flask import Flask, request, jsonify, make_response
```

And after the AuthorSchema, write your first endpoint /authors with the following code:

```
@app.route('/authors', methods = ['GET'])
def index():
    get_authors = Authors.query.all()
    author_schema = AuthorSchema(many=True)
    authors, error = author_schema.dump(get_authors)
    return make_response(jsonify({"authors": authors}))
```

In this method, we are fetching all the authors in the DB, dumping it in the AuthorSchema, and returning the result in JSON.

If you start the application and hit the endpoint now, it will return an empty array since we haven't added anything in the DB yet, but let's go ahead and try the endpoint.

Run the application using Python app.py, and then query the endpoint using your preferred REST client. I'll be using Postman to request the endpoint.

So just open your Postman and GET `http://localhost:5000/authors` to query the endpoint (Figure 2-1).

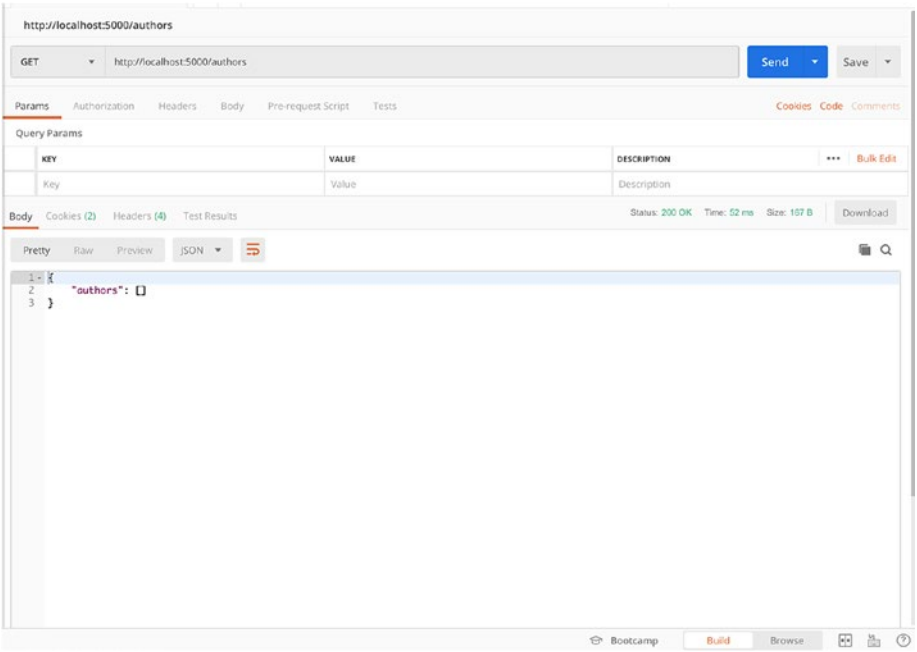

Figure 2-1. *GET /authors response*

You should see a similar result in your Postman client. Now let's create the POST endpoint to add authors to our database.

We can add an object to the table by either directly creating an Authors class in our method or by creating a classMethod to create a new object in Authors class and then calling the method in our endpoint. Let's add the class Method in Authors class to create a new object.

Add the following code in Authors class after fields definition:

```
def create(self):
  db.session.add(self)
  db.session.commit()
  return self
```

The preceding method creates a new object with the data and then returns the created object. Now your Authors class should look like this:

```
class Authors(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(20))
    specialisation = db.Column(db.String(50))

    def create(self):
      db.session.add(self)
      db.session.commit()
      return self

    def __init__(self, name, specialisation):
        self.name = name
        self.specialisation = specialisation
    def __repr__(self):
        return '<Author %d>' % self.id
```

Now we'll create our POST authors endpoint and write the following code after the GET endpoint:

```
@app.route('/authors', methods = ['POST'])
def create_author():
    data = request.get_json()
    author_schema = AuthorsSchema()
    author, error = author_schema.load(data)
    result = author_schema.dump(author.create()).data
    return make_response(jsonify({"author": authors}),201)
```

The previous method will take the JSON request data, load the data in the marshmallow schema, and then call the create method we created in the Authors class which will return the created object with 201 status code.

So let's request the POST endpoint with sample data and check the response. Let's open Postman and POST /authors with JSON request body.

We need to add name and specialisation fields in our body to create the object. Our sample request body should look like the following:

```
{
      "name" : "Kunal Relan",
      "specialisation" : "Python"
}
```

Once we request the endpoint, we shall get Author object in response with our newly created Author. Notice that in this case, the return status code is 201 which is the status code for a new object (Figure 2-2).

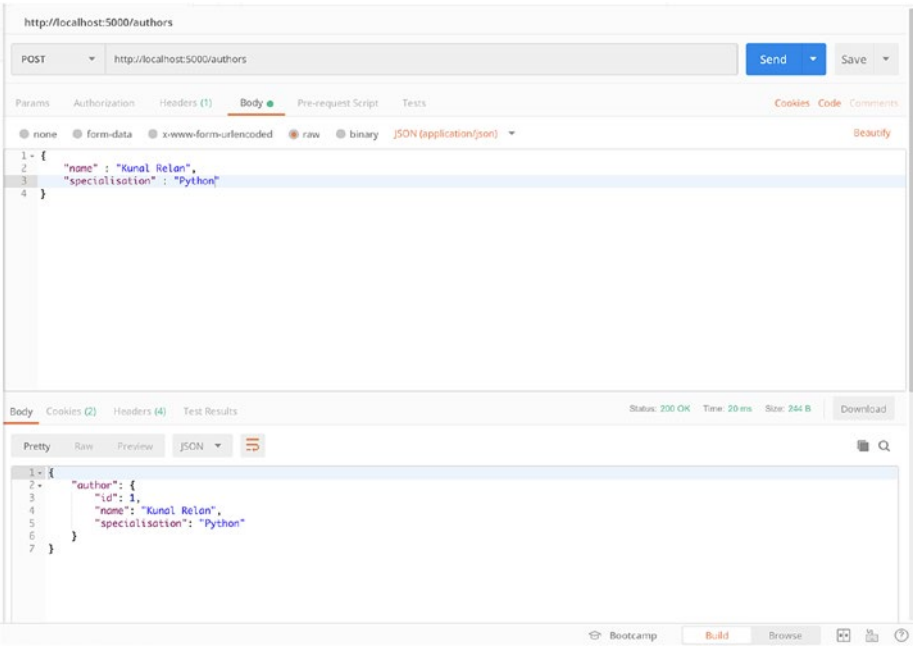

Figure 2-2. *POST /authors endpoint*

So now, if we request our GET /authors endpoint, we shall get our newly created author in the response.

Revisit the GET /authors tab in Postman and hit the request again; this time you should get an array of authors with our newly created Author (Figure 2-3).

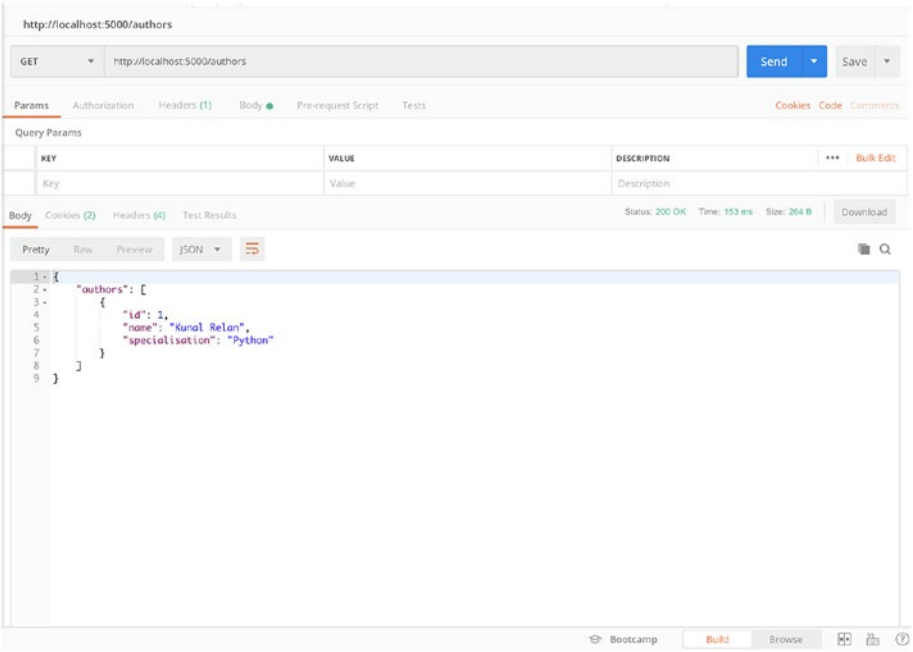

Figure 2-3. *GET all authors with new object*

So far, we have created endpoints to register new authors and to fetch a list of authors. Next we'll create an endpoint to return author using the author ID and then update endpoint to update author details using author ID and the last endpoint to delete an author using author ID.

For GET author by ID, we'll have a route like /authors/<id> which will take author ID from the request parameter and find the matching author.

Add the following code for the GET author by ID endpoint below your GET all authors route.

```
@app.route('/authors/<id>', methods = ['GET'])
def get_author_by_id(id):
```

```
    get_author = Authors.query.get(id)
    author_schema = AuthorsSchema()
    author, error = author_schema.dump(get_author)
    return make_response(jsonify({"author": author}))
```

Next we need to test this endpoint, and we'll request for author with ID 1 as we see in the preceding GET all authors API response, so let's open Postman again and request /authors/1 on our application server to check the response.

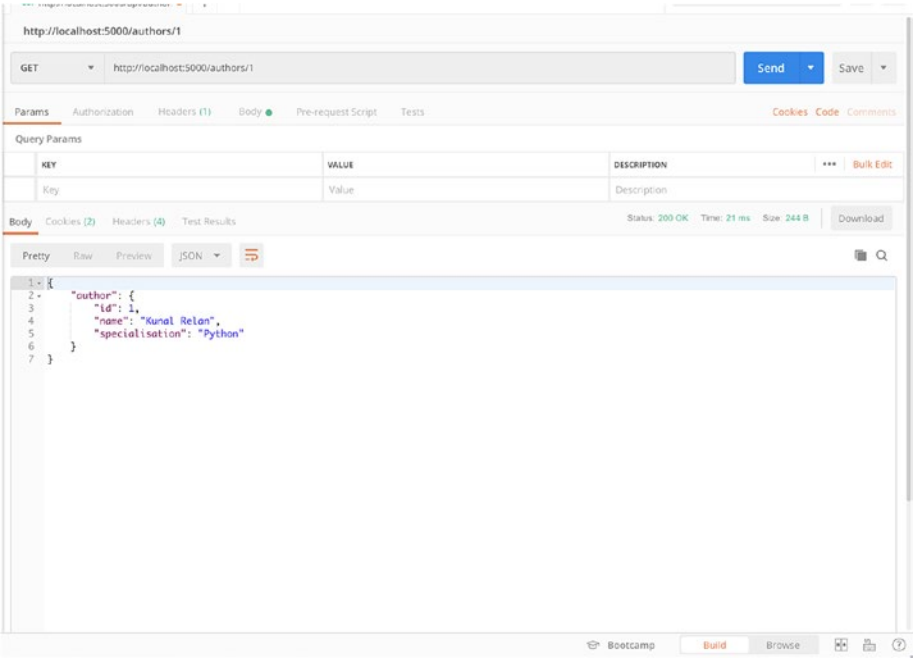

Figure 2-4. *GET author by ID endpoint*

As you see in the preceding screenshot, we are returning an object with a key author containing the author object with ID 1. You can now add more authors using the POST endpoint and fetch them using the returned ID.

Next, we need to create an endpoint to update the author name or specialisation, and for updating any object, we'll use PUT HTTP verb as we discussed in the "Introduction to RESTful Services" section. This endpoint will be similar to the GET authors by ID endpoint but will use PUT verb rather than the GET one.

Here is the code for the PUT endpoint to update an author object

```
@app.route('/authors/<id>', methods = ['PUT'])
def update_author_by_id(id):
    data = request.get_json()
    get_author = Authors.query.get(id)
    if data.get('specialisation'):
        get_author.specialisation = data['specialisation']
    if data.get('name'):
        get_author.name = data['name']
    db.session.add(get_author)
    db.session.commit()
    author_schema = AuthorsSchema(only=['id', 'name',
    'specialisation'])
    author, error = author_schema.dump(get_author)
    return make_response(jsonify({"author": author}))
```

So let's test our PUT endpoint and change the specialisation of author ID 1.

We'll PUT the following JSON body to update the author specialisation.

```
{
      "specialisation" : "Python Applications"
}
```

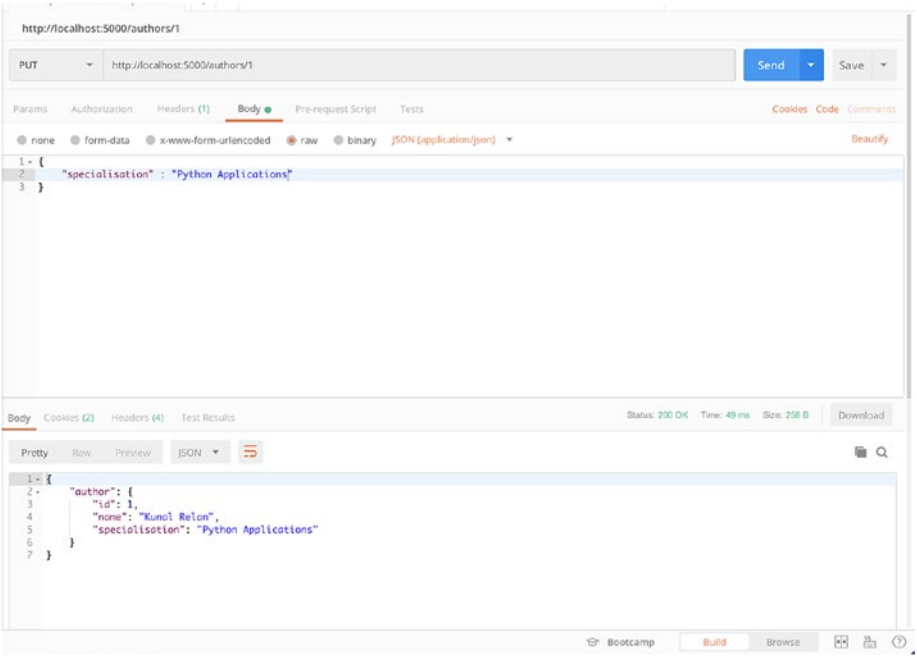

Figure 2-5. *UPDATE author by ID endpoint*

As you can see in Figure 2-5, we updated the author with ID 1, and now the specialisation has been updated to "Python Applications".

Now, the last endpoint to remove an author from the database. Add the following code to add a delete endpoint which will look like get author by ID endpoint but will use DELETE verb and return 204 status code with no content.

```
@app.route('/authors/<id>', methods = ['DELETE'])
def delete_author_by_id(id):
    get_author = Authors.query.get(id)
    db.session.delete(get_author)
    db.session.commit()
    return make_response("",204)
```

And now we'll request the delete endpoint to remove our author with ID 1 (Figure 2-6).

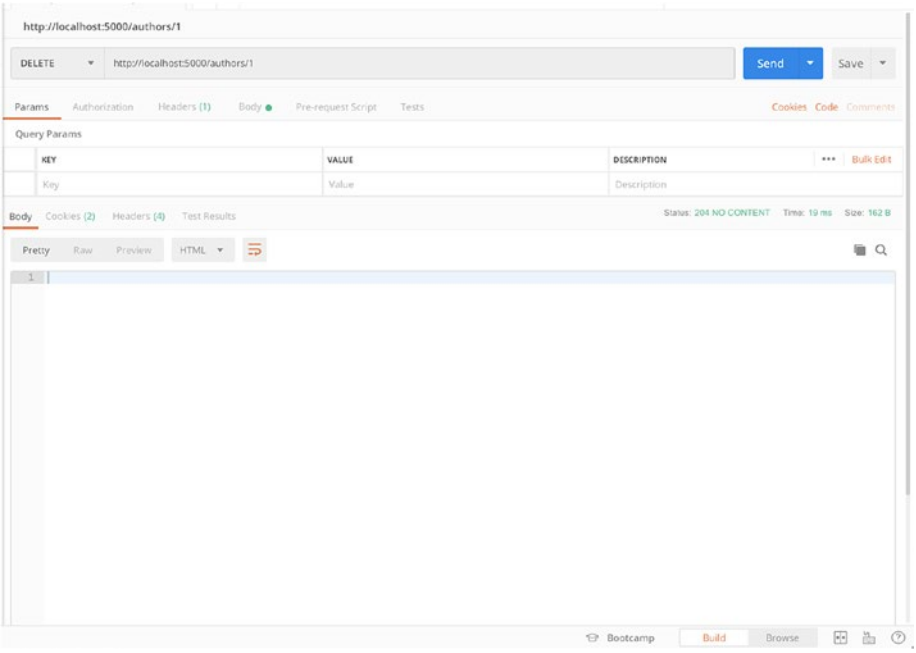

Figure 2-6. *DELETE author by ID*

And now if you request GET all authors endpoint, it shall return an empty array.

Now your app.py should have the following code:

```
from flask import Flask, request, jsonify, make_response
from flask_sqlalchemy import SQLAlchemy
from marshmallow_sqlalchemy import ModelSchema
from marshmallow import fields

app = Flask(__name__)
app.config["SQLALCHEMY_DATABASE_URI"] = 'mysql+pymysql://<mysql_
username>:<mysql_password>@<mysql_host>:<mysql_port>/<mysql_db>'
```

```
db = SQLAlchemy(app)

class Authors(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(20))
    specialisation = db.Column(db.String(50))

    def create(self):
      db.session.add(self)
      db.session.commit()
      return self

    def __init__(self, name, specialisation):
        self.name = name
        self.specialisation = specialisation

    def __repr__(self):
        return '<Author %d>' % self.id

db.create_all()

class AuthorsSchema(ModelSchema):
    class Meta(ModelSchema.Meta):
        model = Authors
        sqla_session = db.session

    id = fields.Number(dump_only=True)
    name = fields.String(required=True)
    specialisation = fields.String(required=True)

@app.route('/authors', methods = ['GET'])
def index():
    get_authors = Authors.query.all()
    author_schema = AuthorsSchema(many=True)
    authors, error = author_schema.dump(get_authors)
    return make_response(jsonify({"authors": authors}))
```

```
@app.route('/authors/<id>', methods = ['GET'])
def get_author_by_id(id):
    get_author = Authors.query.get(id)
    author_schema = AuthorsSchema()
    author, error = author_schema.dump(get_author)
    return make_response(jsonify({"author": author}))

@app.route('/authors/<id>', methods = ['PUT'])
def update_author_by_id(id):
    data = request.get_json()
    get_author = Authors.query.get(id)
    if data.get('specialisation'):
        get_author.specialisation = data['specialisation']
    if data.get('name'):
        get_author.name = data['name']
    db.session.add(get_author)
    db.session.commit()
    author_schema = AuthorsSchema(only=['id', 'name',
    'specialisation'])
    author, error = author_schema.dump(get_author)
    return make_response(jsonify({"author": author}))

@app.route('/authors/<id>', methods = ['DELETE'])
def delete_author_by_id(id):
    get_author = Authors.query.get(id)
    db.session.delete(get_author)
    db.session.commit()
    return make_response("",204)

@app.route('/authors', methods = ['POST'])
def create_author():
    data = request.get_json()
    author_schema = AuthorsSchema()
```